Cuboid

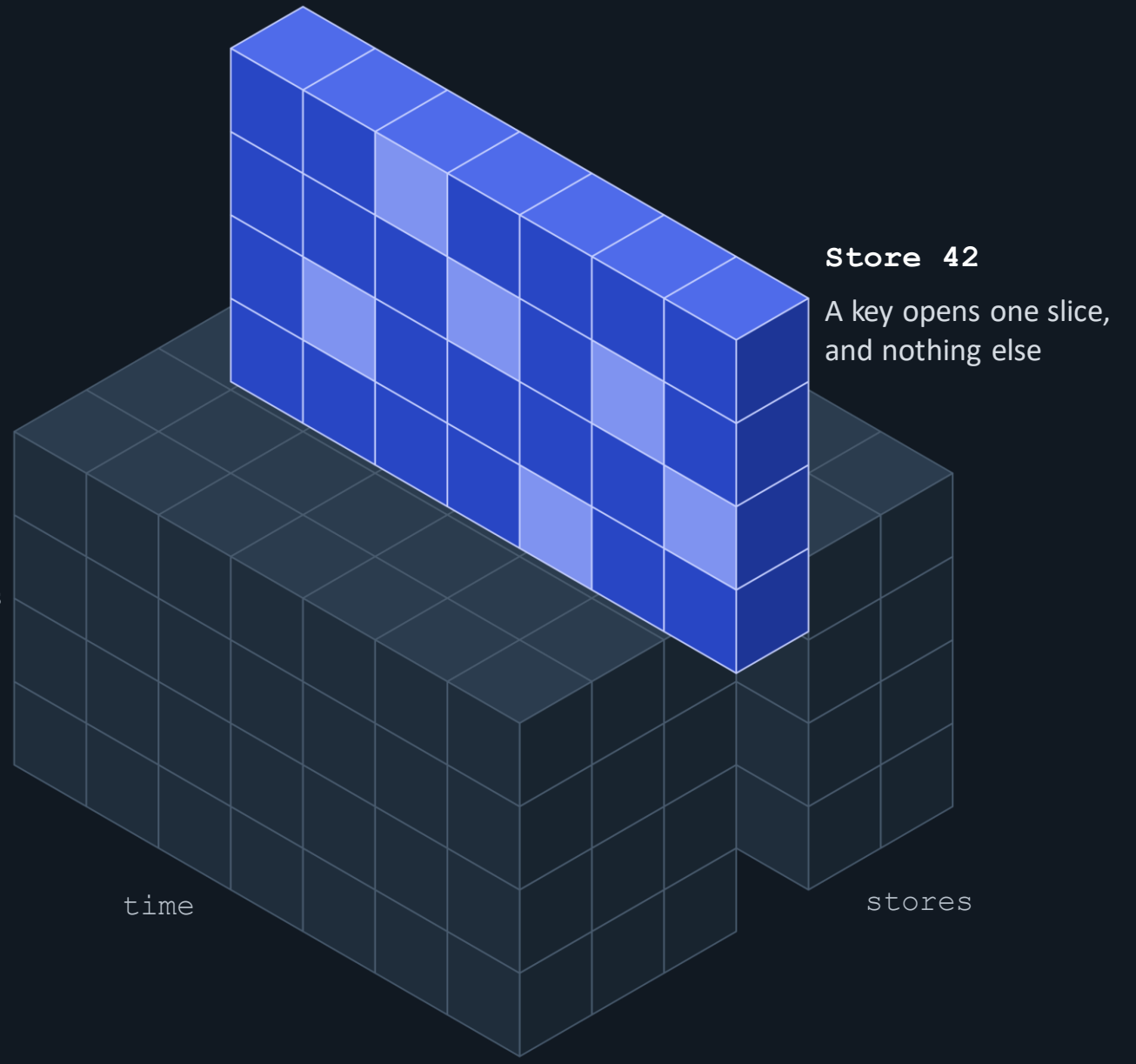
An aggregation layer between the database and the dashboards: fast, fresh and exact for every store, and sealed off from every other.

10M orders

100 stores

1,000 dashboards

1 laptop



We reproduced the problem before solving it

Is slow

“it scans lots of data, so the dashboard takes seconds or minutes to load”

5.4 s

Dashboard p99 at 10M orders with 1,000 dashboards querying directly, on properly indexed tables.

Hurts everyone

“one big store’s report makes the app slow for all stores”

105×

The store app’s own p95 went from 0.99 ms to 104 ms. Its throughput fell from 188 to 54 operations a second.

Forces bad shortcuts

“a nightly job is cheap but shows yesterday’s numbers; querying live is fresh but expensive”

24 h

That is the choice: numbers up to a day old, or the slowdown beside this. We wanted neither.

So Cuboid never asks the main database for a dashboard number at all.

Evidence: artifacts/safety/main.json · Direct phase, 1,000 dashboards on the main database, 10M orders

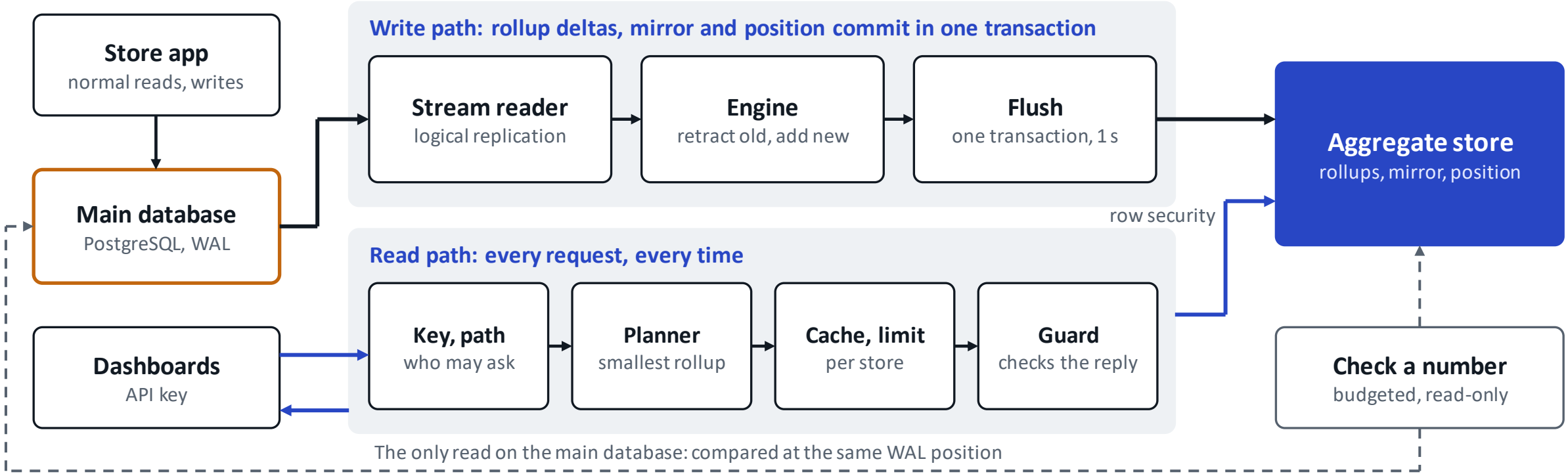
Five goals met; one query misses the sixth

PRD goal	How the PRD measures it	Good	Great	Measured at 10M orders	Verdict
Fast	Time for each dashboard query	Under 1 s	Under 200 ms	1,000 dashboards: p99 3 ms. Slowest uncached query: 230 ms	Good
Fresh	Adding an order to seeing it in the numbers	Under 5 min	Under 30 s	p99 1.1 s, for inserts and updates	Great
Safe	Normal app query time, with and without dashboard load	Under 25%	Under 10%	+17.5%: p95 0.99 ms to 1.16 ms	Good
Separate	A tenant only ever sees its own data	No band	No band	0 of 1,000,000 requests crossed stores. 0 leaked in drills	Met
Correct	Compare with a direct database query	Most queries	All queries	2,500 of 2,500 exact, 500 while writes ran	Great
Scales	Query speed at 1M vs 10M records	Under 5x	Under 2x	7 of 8 queries at 3.2x or less. Top customers: 13.7x	Not met

Read it fairly: seven of eight queries beat Great on speed; viewers get even the eighth from cache in under 2 ms. Against Cuboid stopped entirely, safety is +28.4%, a quarter of a millisecond on one shared laptop.

Evidence: docs/TARGETS.md · artifacts/ · measured at 10M orders on one laptop, 12 logical CPUs, 15.6 GB of RAM

Dashboards read a copy built from the stream



WHAT THE PRD SAID WE WOULD LEARN

Indexing
Cells keyed by store, rollup, grain and bucket

Pre-computing
Exact rollups that retract under updates and deletes

Caching
One partition per store, valid by time range

Streams
Logical replication with a one-second flush

Measuring
Every number written to a file in artifacts/

Evidence: docs/ARCHITECTURE.md · docs/adr/ · Go, PostgreSQL 16, React · no polling and no triggers on the main database

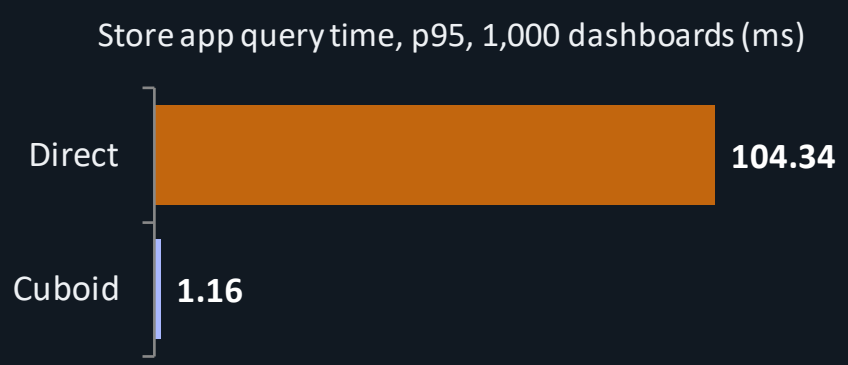
Every must-have, through one simple API

ID	The PRD's example	The real GET, largest store	Result at 10M
FR-1	Orders and total sales, last 7 days	`/tenants/42/orders/summary?range=last_7d`	**5.7 ms**
FR-2	Orders per day, last 30 days	`/tenants/42/orders/trend?interval=day&range=last_30d`	**8.8 ms**
FR-3	Top 10 products by sales this week	`/tenants/42/products/top?n=10&by=sales&range=this_week`	**38.6 ms**
FR-4	Orders by status	`/tenants/42/orders/breakdown?by=status&range=last_7d`	**6.9 ms**
FR-5	Store A can never see Store B's numbers	Store 42's key on `/tenants/43/orders/count`	**404, same as a missing store**
FR-6	Add an order and see the count go up; placed to cancelled	POST an order, PATCH it to cancelled, GET the breakdown	**About 1.0 s, both ways**
FR-7	Simple API, short README of the endpoints	`/tenants/42/orders/count?from=...&to=...`	**README, OpenAPI, Postman**

Company owners get the same calls under /companies/{id}, across their stores. **Data:** 10M orders across 100 stores, the largest with 19.3%, from our generator, since the organisers' script had not arrived.

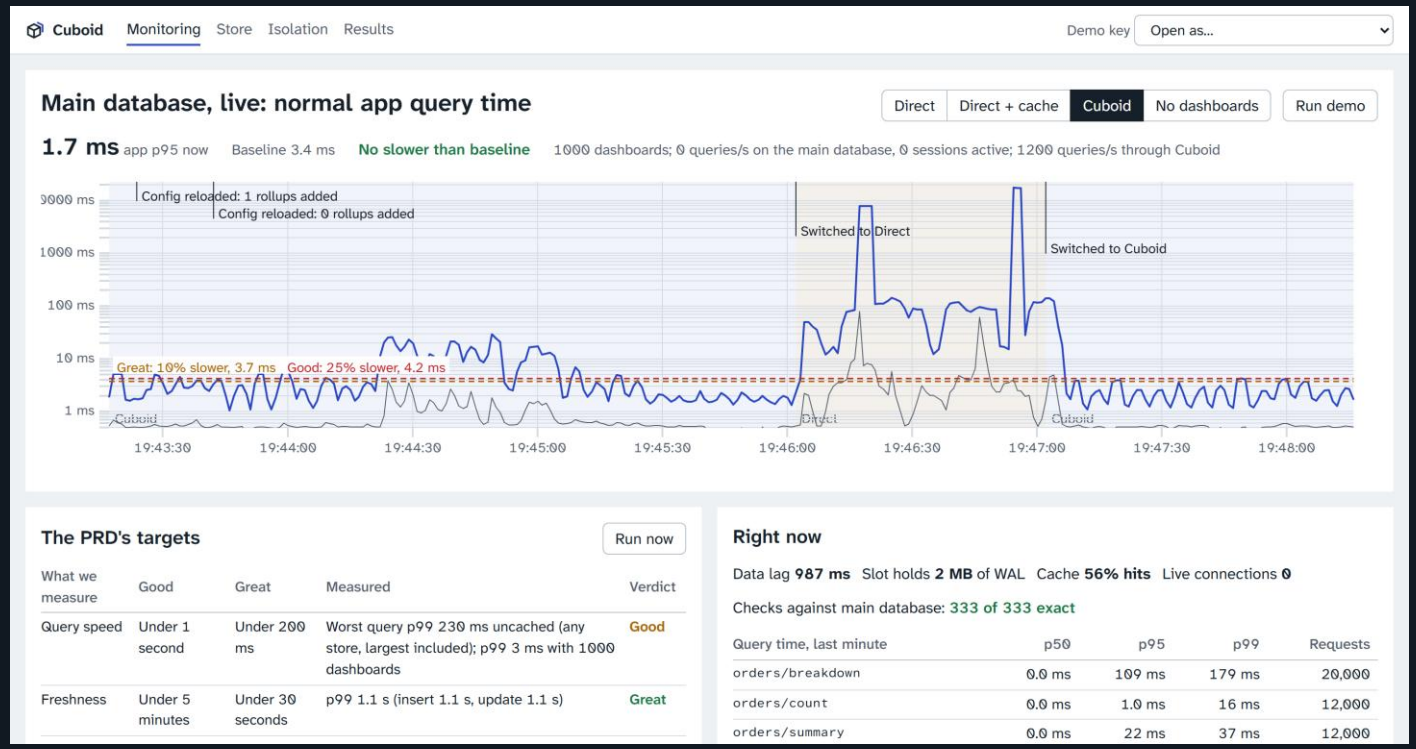
Evidence: artifacts/speed/main.json (p99, uncached, largest store) · artifacts/demo-check/main.json: 23 of 23 pass at 10M

Same load, same database: 104 ms to 1.16 ms



5.7 ms dashboard p99 through Cuboid. Direct: 5.4 s

184 /s app operations with 1,000 dashboards. Direct: 54. None: 188

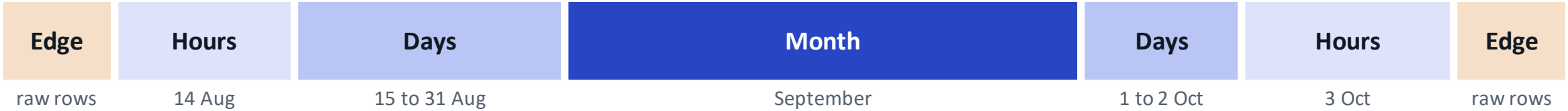


Measuring found a real bug. The first full run at this size showed dashboards at p99 1.4 s and the app 412% slower: every one-second flush was recomputing every moving range. Refreshes now back off under load, and viewers get the last exact answer labelled with its true age. Result: p99 3 ms, and +17.5%.

Evidence: artifacts/safety/main.json, 120-second phases · artifacts/speed/main.json · screenshot from this stack under real load

Fresh in a second, exact at any timestamp

14 August 09:17:03 to 3 October 18:41:59, read as:



How a number is proved

1. **Fence**
   Write a marker into the WAL, and wait for Cuboid to reach it.
2. **Compare**
   Read Cuboid's answer, and run direct SQL on main at that point.
3. **Reconcile**
   Account for transactions still in flight, so a match means exact.

1.1 s

p99, new order to visible, and updates too. None of 400 probes timed out.

2,500 / 2,500

exact against direct SQL, with arbitrary timestamps. 500 ran during writes.

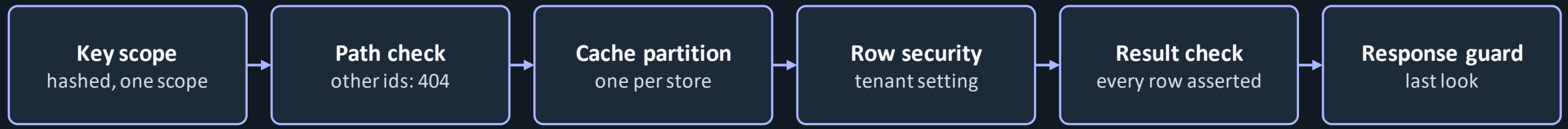
The check found what unit tests could not

Two orders in three minutes went missing. A commit right after a heartbeat shares its log position, so it looked already applied.

Fixed, with a test. Only comparing with the live database could show it.

Evidence: artifacts/freshness/main.json · artifacts/correctness/main.json · internal/pipeline/position_test.go

We inject INC-02 and count what leaks



Drill A: shared cache key. 538,506 faults — stopped at the guard

Drill B: connection mix-up, the INC-02 mechanism. 20,452 faults — stopped at the result check

Drill C: missing tenant setting. 21,514 faults — stopped at row security

0 leaked

in all three drills at 10M orders, while requests were cancelled mid-flight

0 of 1,000,000

randomized requests crossed stores, 57,153 of them cancelled by the client

20 kills

kill -9 under writes: back within 3.1 s, and 500 of 500 checks exact

Six checks, each assuming the others failed. 30 of 30 red-team checks also pass: bad keys, injection, row security, recategorising, order moves.

Evidence: artifacts/{demo-check,isolation,chaos,redteam}/main.json · docs/ISOLATION.md

All five bonuses, each shown live

B-1 **How fresh**

Every response states its age and log position, and stale answers say so.

B-2 **Rebuild from scratch**

From the mirror and from main at 10M: 5,395 queries answered during them, 0 failed.

B-3 **Limit heavy queries**

The biggest store floods: 3,560 limited. Other stores' p99 stayed under 3 ms.

B-4 **Metric by config**

One line of YAML: live in 44 s at 10M, with no restart and no main-database load.

B-5 **Monitoring page**

Query time, data lag, target lines, limiter, and every control in one place.

config/metrics.yaml: one line, B-4

```
status_payment: {source: orders,
  dims: [status, payment_method],
  grains: {hour: 35d, day: all, month: all}}
```

every response: B-1

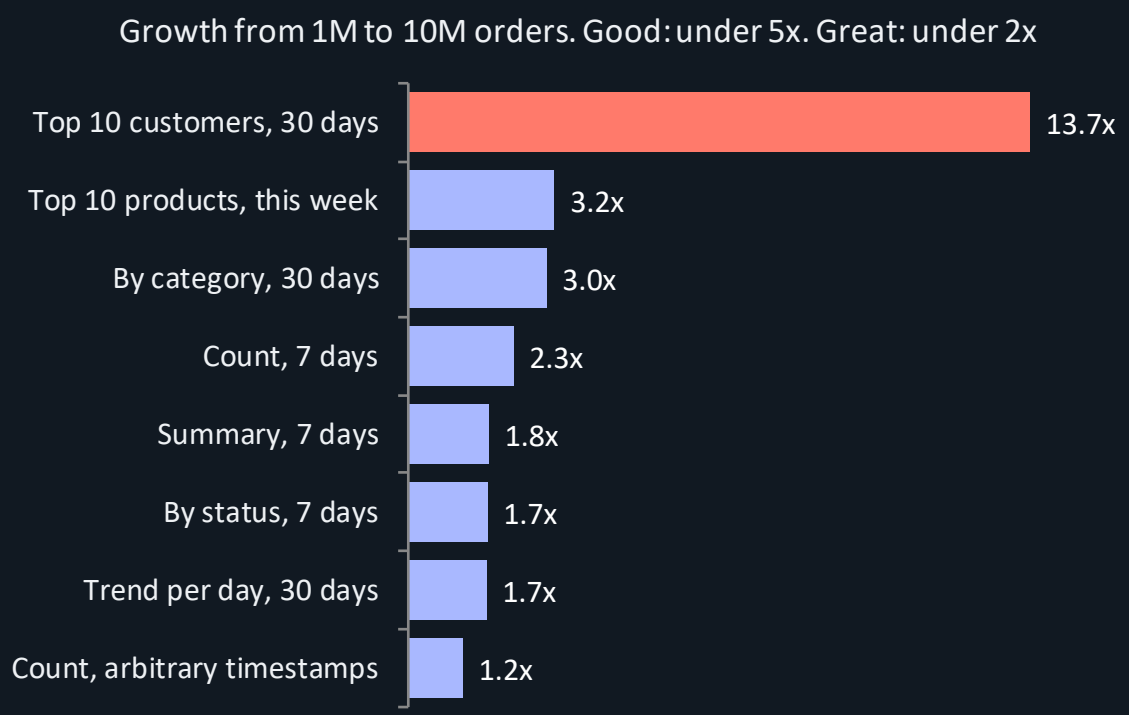
```
"freshness": {"lsn": "3/6EFAE260",
  "lag_ms": 1107, "stale": false,
  "updated": "updated 1 second ago"}
```

Beyond the brief

- **Same engine, Mitigata's data:** SOC alerts through config alone
- **Run now:** re-measures speed, freshness, correctness live
- **Autopilot:** the demo, through real APIs, 3 clean runs
- **Company view:** all an owner's stores, and each one

Evidence: artifacts/demo-check/main.json D11 to D15, on the 10M tier · config/metrics.yaml · web/src/views/Monitor.tsx

Where it falls short, said before you ask



Top customers is the limit.

Its cost grows with the customers in the range. The designed fix, running window totals, touches the exactly-once path, so we did not rush it.

Docker is written, not run.

Docker Desktop broke on our machine. All numbers come from the native path, with the same Postgres version, settings and ports.

The organisers' script never arrived.

We built a generator to the PRD's shape. Switching is documented, not proven.

One laptop, 100 tenants.

The path to 1,000 tenants is a design, not yet built.

Fast for every store. Exact to the paisa. Never in the wrong store's hands.

github.com/WarHawkADI/cuboid

Evidence: artifacts/growth/latest.json, largest store, uncached p99; 1M figures are from an earlier build · docs/FINAL_REPORT.md